Act as a Stoquify multidisciplinary principal engineering, product, controls, and operations review board. Cover enterprise/platform architecture; backend, API, and distributed systems; database, data integrity, and migration engineering; application security, IAM/RBAC, privacy, fraud, and abuse prevention; frontend and design-system engineering; workflow/service UX, accessibility, localization, and content design; product strategy and business-process analysis; finance, accounting, reconciliation, and internal controls; OHADA/SYSCOHADA statutory and country-pack compliance; quality engineering and release assurance; SRE, DevSecOps, observability, resilience, performance, and cost; integration, event-driven, offline/edge, and provider-boundary architecture; analytics and data governance; AI/agent safety, evaluation, and human-approval governance; and SaaS modularity, packaging, billing, growth, customer-success, and product-operations strategy.

Operate as one coordinated team. Make evidence-backed recommendations, expose disagreements and tradeoffs, trace impacts across UX, services, data, controls, infrastructure, operations, and commercial packaging, and distinguish current repository truth from proposals. Use every applicable lens without widening a narrow request into an unrelated rewrite. Mark immaterial lenses `not applicable` with one short reason. Never claim legal, tax, accounting, security, accessibility, privacy, or release certification without the required expert-reviewed evidence.

Project: Stoquify Workspace: E:\ohada saas\Focused projects\stoquify Mission: End-to-end accounting integrity, architecture, integration, performance, security, and user-experience audit.

1. Objective and scope

Review the accounting domain from database tables and migrations through domain services, server actions, APIs, events, TanStack Query hooks, and rendered UI components.

Determine whether the implementation supports trustworthy accounting, reliable daily operations, efficient financial reporting, and the professional standard expected of an enterprise accounting platform.

Trace every material accounting connection with POS, sales, purchasing, inventory, finance, payments, reconciliation, payroll, taxation, compliance, reporting, and period close.

Produce a comprehensive analysis explaining:

- What is implemented correctly and should be preserved.
- What is incomplete, inconsistent, fragile, inefficient, or incorrect.
- What is missing, intentionally deferred, or impossible to verify.
- What must be hardened, in what order, and why.
- The principal financial, operational, security, statutory, and reputational risks if confirmed weaknesses remain unresolved.

This task is an audit and remediation plan. Write reports and diagnostic artifacts; do not implement production changes, alter business records, apply migrations, reseed databases, or deploy changes.

2. Review responsibilities

Use the permanent core reviewers: enterprise/platform architect; backend/domain/integration engineer; database/migration architect; application-security/IAM/privacy architect; frontend/design-systems engineer; workflow/accessibility/localization specialist; product/business-process analyst; quality/release-assurance lead; SRE/DevSecOps/performance engineer; and SaaS packaging/billing/product-operations strategist.

Also activate domain review for accounting and internal controls; OHADA/SYSCOHADA and country packs; audit evidence and records governance; POS and offline synchronization; purchasing/AP; payroll; payments and

reconciliation; close and accountant access; analytics; AI/agent boundaries; APIs/webhooks/events; and operational rollout/support.

Record an evidence-backed finding, a verified strength, or an explicit `not applicable` decision for each responsibility. Use these perspectives to examine accounting and its dependencies without expanding into unrelated product redesign.

3. Evidence-first discovery

Read applicable AGENTS.md instructions and record the current commit, branch, and relevant uncommitted changes. Preserve the existing worktree.

Inspect:

- Current accounting routes, components, hooks, types, actions, services, APIs, workers, integration adapters, configuration, permissions, tests, and scripts.
- E:\ohada saas\Focused projects\stoquify\prisma\schema.prisma and relevant migration SQL.
- E:\ohada saas\Focused projects\stoquify\types\queryKeys.ts
- E:\ohada saas\Focused projects\stoquify\hooks\useAccountingControlCenter.ts
- E:\ohada saas\Focused projects\stoquify\hooks\useAccountantPortal.ts
- E:\ohada saas\Focused projects\stoquify\what-next\accounting-financial-reporting-suite-implementation-2026-08-01.md
- Relevant accounting, outbox, POS-ledger, reconciliation, close-assurance, and readiness reports under the workspace's what-next, innovation, and docs directories.
- Existing graphify-out graphs and reports for actions, routes, components, hooks, and types.

Use graph nodes and communities to navigate relationships, then verify important relationships against current source. Treat historical reports, inferred graph edges, and passing historical gates as leads rather than proof of current correctness.

Identify active, legacy, duplicate, and bypass paths, including direct database access outside the intended accounting service boundary.

Distinguish schema declarations, committed migrations, and verified deployed database constraints. Do not assume deployment state from repository files.

4. Map the complete accounting system

Create an inventory of accounting capabilities, tables, services, entry points, read models, queries, screens, reports, and integration producers.

For each material workflow, trace:

Business event → source record → authorization → action/API → domain service → transaction/outbox → ledger/subledger → reporting read model → TanStack Query → UI/export → reconciliation/close evidence.

Document:

- Authoritative source and owning service.
- Relevant tenant, entity, location, currency, and period scope.
- State transitions and approvals.
- Transaction boundaries and asynchronous handoffs.

- Idempotency, retry, ordering, and recovery behavior.
- Report dependencies and cache invalidation.
- Audit evidence and operational ownership.

Include Mermaid diagrams for the accounting architecture and critical cross-module flows. Identify every material producer and consumer; sample runtime scenarios explicitly rather than implying exhaustive execution.

5. Verify accounting correctness and database integrity

Examine whether the system enforces:

- Balanced journals, valid posting accounts, valid debit/credit lines, and appropriate control-account restrictions.
- Correct account classification, posting templates, journal types, and mapping completeness.
- Monetary precision throughout database storage, arithmetic, serialization, aggregation, and display.
- Explicit currency, exchange-rate provenance, rounding, and currency-specific precision where supported.
- Correct accounting dates, fiscal periods, cutoffs, time zones, opening balances, carry-forwards, and year-end transitions.
- Controlled draft, approval, posting, reversal, correction, close, and reopening lifecycles.
- Protection against changing or deleting posted financial history, including through alternate endpoints, scripts, or cascading deletes.
- Source-document traceability, unique posting identity, referential integrity, and durable audit history.
- Appropriate constraints, indexes, concurrency controls, and migration/backfill safeguards.

Balanced entries alone do not establish correctness. Verify economic substance, recognition timing, completeness, account selection, and reconciliation to the originating transaction.

Trace actual reversal semantics: establish how original entries and reversing entries contribute to reports, including entries marked REVERSED. Check for omission, cancellation errors, and double counting.

For unsupported capabilities, state whether they are outside the advertised scope, safely blocked, misleadingly exposed, or required for the intended accounting workflows.

6. Reconcile accounting with connected modules

Build a cross-module matrix covering at least:

- POS and sales: cash/credit sales, invoices, discounts, taxes, split tenders, refunds, voids, returns, cash-drawer variances, and customer receivables.
- Purchasing/AP: purchase orders, partial receipts, supplier invoices, receipt/invoice timing differences, matching exceptions, credits, returns, supplier balances, and payments.
- Inventory: receipts, issues, transfers, reservations versus ownership, valuation, cost of goods sold, adjustments, write-offs, negative stock, backdated movements, and landed costs where supported.
- Finance and treasury: cash/bank movements, internal transfers, expenses, payment allocation, settlement, fees, suspense, and reconciliation.
- Payroll: approved payroll liabilities, expenses, statutory deductions, payments, corrections, and close dependencies.
- Tax/compliance: tax bases, collected/recoverable tax, exemptions, rounding, fiscal documents, declarations, and country-specific mappings.

- Reporting/close: trial balance, general ledger, subsidiary ledgers, statements, close blockers, evidence packs, and accountant access.

For each flow specify:

- Trigger, source record, expected accounting effect, and applicable policy.
- Actual implementation and validation evidence.
- Timing differences that are legitimate versus unexplained discrepancies.
- Reconciliation mechanism, exception owner, and recovery route.
- Behavior under duplicates, concurrent requests, partial failure, offline replay, out-of-order events, and late corrections.

Check module activation, suspension, deactivation, and permission changes for their effects on pending postings, historical access, and reconciliation.

Verify that operational dashboards and sales/cash analytics are not presented as posted-ledger accounting results without explicit reconciliation and labeling.

7. Review services, actions, and distributed reliability

Examine business-rule ownership, input validation, server-derived scope, transaction isolation, atomic writes, retry policies, and error contracts.

Check whether source transactions can succeed while accounting silently fails, or accounting can post while source operations fail. Establish whether such states are prevented or durably tracked and recoverable.

Inspect outbox delivery, consumer deduplication, posting identities, replay tools, dead-letter handling, and reconciliation jobs. Verify one financial effect per intended event despite repeated delivery; do not infer this guarantee from “exactly once” terminology.

Identify swallowed exceptions, success responses hiding incomplete accounting, unsafe fallback values, orphaned postings, and ambiguous commit outcomes.

Review logs, metrics, alerts, queue age, unposted-event monitoring, reconciliation discrepancies, recovery procedures, and close invalidation after material changes.

8. Review access, controls, and evidence

Verify tenant/entity isolation on reads, writes, aggregates, exports, background jobs, and cached responses.

Inspect RBAC, module entitlements, fresh authorization for sensitive operations, least privilege, and maker-checker separation where required by policy.

Test direct endpoint access and alternate paths independently of hidden or disabled UI controls.

Review privileged overrides, journal adjustments, period reopening, account-mapping changes, bulk imports, and exports for authorization and traceability.

Check audit provenance, actor identity, evidence retention, redaction, tamper resistance, and safe error disclosure. Include accountant-portal and AI/agent access where connected to accounting.

9. Review TanStack Query and UI correctness

Trace query keys, query functions, DTOs, mutation handlers, server caching, and UI consumers.

Verify:

- Tenant/entity, period, currency, filters, and other result-changing inputs are represented in cache identity.
- Organization switches, logout, permission changes, and entitlement changes cannot expose stale protected data.
- Posting, reversal, payment, reconciliation, and closing invalidate or refresh every affected financial view.
- Retry and optimistic-update behavior cannot duplicate financial operations or present uncommitted results as final.
- Loading, empty, denied, failed, stale, incomplete, and genuinely zero-value states remain distinguishable.
- Decimal values, dates, signs, totals, pagination, and exported results preserve server meaning.

Assess complete workflows for accountants, owners, finance operators, approvers, and auditors.

Review information hierarchy, dense-table usability, keyboard operation, accessibility, responsive behavior, French/English localization, actionable validation, recovery, and prevention of costly user errors.

Confirm users can distinguish draft versus posted, reconciled versus unresolved, and open versus closed periods; identify report scope and freshness; and drill from totals to journal lines and source evidence.

Support UI findings with browser evidence where available. Label static-only review and blocked runtime access explicitly. Recommend modern presentation changes only when they improve clarity, accessibility, efficiency, or control.

10. Verify reporting, statutory boundaries, and performance

Reconcile trial balance and subsidiary ledgers to the general ledger and source records.

Review period activity versus cumulative balances, opening balances, year-end transfers, profit/loss carry-forward, report signs, account classifications, contra accounts, comparatives, and unclassified balances.

Verify balance-sheet equality and explain legitimate differences between accrual earnings and cash movements. Check management EBITDA presentation separately from statutory statements.

Evaluate financial statements, cash-flow reporting, exports, and close packs against their actual advertised scope. Identify incomplete capabilities without treating every deferred feature as a defect.

For OHADA/SYSCOHADA or country-specific claims, verify jurisdiction, entity regime, effective dates, configuration versions, authoritative sources, and qualified review evidence. Separate technical enforcement from professional accounting or statutory validation.

Measure material performance paths where feasible: large ledgers, long date ranges, concurrent posting, close calculations, dashboard queries, and exports.

Inspect query plans, indexes, N+1 queries, unbounded reads, aggregation strategy, locking, pagination, payload size, rendering cost, and cache freshness. Separate measured bottlenecks from hypotheses; preserve accounting consistency in every optimization proposal.

11. Execution checklist and verification commands

Proceed through discovery, architecture mapping, invariant review, cross-module tracing, focused verification, risk analysis, and remediation planning.

Inspect package.json and script bodies before executing checks. Existing scripts include:

```
npm run prisma:validate
npm run typecheck
npm test -- --runInBand --runTestsByPath <verified-test-paths>
npm run ledger:close-truth:gate
npm run statutory:country-pack:integration:gate
```

Use focused existing tests for accounting, source-to-ledger integration, reversals, authorization, concurrency, reporting, cache invalidation, and close behavior.

Run only understood, appropriate checks. The current test:e2e:close-assurance script chains migration deployment and seeding; do not invoke that composite script as a read-only audit check. Use an isolated disposable test environment for database-writing scenarios.

Preserve existing evidence files; direct generated reports to the audit folder when supported.

Record exact commands, environment, timestamp, result, and evidence location. Distinguish passed, failed, skipped, timed out, and blocked checks. Explain what each check proves and what remains unverified.

12. Findings, risks, and remediation plan

For every finding record:

- Stable ID, affected capability, and status.
- Severity, confidence, likelihood, and impact.
- Exact source locations, relevant symbols, and runtime/test evidence.
- Triggering scenario and expected versus observed behavior.
- Root cause and affected cross-module dependencies.
- Financial consequences, including potential misstatement or unreconciled balances.
- Operational, fraud, security, statutory, and reputational consequences.
- Existing mitigating controls.
- Recommended correction, owner, dependencies, effort range, and acceptance criteria.
- Data investigation, repair, backfill, migration, and rollout needs where applicable.

Distinguish confirmed defects, credible risks needing validation, missing controls, intentional limitations, and external evidence blockers. Do not invent financial loss estimates or present plausible failures as observed incidents.

Rank the most serious risks in plain business language. Explain how each could occur, how far it could propagate, whether it would be detected, and how difficult recovery would be.

Create a dependency-ordered remediation plan:

- Immediate containment for demonstrated material exposure.
- Accounting/data-integrity and authorization corrections.
- Integration, reconciliation, and recovery hardening.
- Performance and workflow improvements.
- Remaining validation and release-readiness evidence.

Include verified strengths and explain why they are trustworthy. Avoid generic enterprise checklists without findings grounded in this repository.

13. Expected artifacts

Save results under a new run folder:

E:\ohada saas\Focused projects\stoquify\what-next\accounting-enterprise-audit<YYYY-MM-DD-run-id>\

Produce:

- audit-report.md: executive assessment, verified strengths, detailed findings, readiness by capability, and outstanding uncertainty.
- architecture-and-flow-map.md: diagrams, source ownership, integration contracts, and cross-module reconciliation matrix.
- findings.json: structured findings with severity, confidence, evidence, and remediation.
- remediation-plan.md: prioritized work packages, dependencies, owners, acceptance criteria, and validation requirements.
- evidence-index.md: commands, results, source references, screenshots, and environment limitations.
- Supporting logs and screenshots when collected.

Conclude with a concise account of what is trustworthy today, what should be restricted or corrected based on evidence, the greatest unresolved risks, and the first concrete actions.

14. Risk controls, non-goals, and success criteria

Preserve tenant boundaries, financial records, secrets, posted history, existing work, and audit evidence. Use synthetic or redacted data in artifacts.

Do not perform unrelated refactors, lint cleanup, framework upgrades, speculative feature development, destructive database operations, or cosmetic redesign.

The audit is complete when:

- Every material accounting capability and integration has a recorded disposition.
- Critical flows are traced from source through ledger, reporting, query state, and UI.
- Correctness claims and significant findings have traceable evidence.
- Important failure paths and reconciliation invariants have been tested or explicitly marked unverified.
- Strengths, defects, limitations, and unknowns are clearly separated.
- The largest risks and remediation dependencies are understandable to both engineering and finance leadership.
- Recommended work is concrete enough for subsequent implementation and verification.
- No readiness or certification claim exceeds the available evidence.